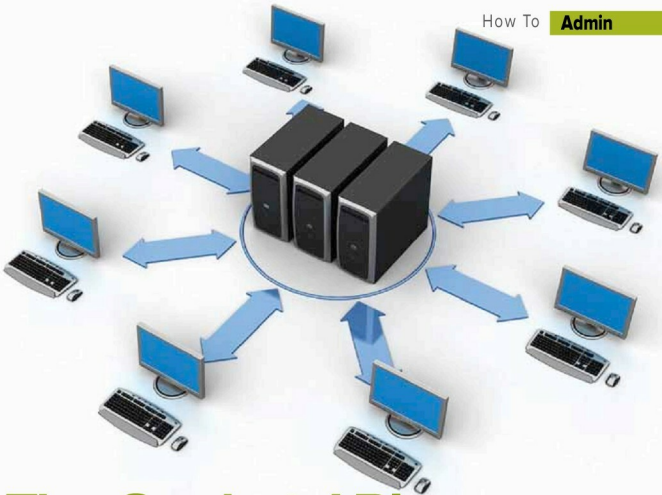




The Socket API

Concurrent Servers

In this part of the series, we will learn how to deal with multiple clients connected to the server.

Part—3

Welcome to another dose of socket programming! Till now, we've created servers that are capable of creating connections with multiple clients, but the problem is that the server will communicate with only one client at any point in time. This is because there is only one socket descriptor, *cfid*, created to communicate with a client—and all connections will wait on the same descriptor. Now, let's use the *fork()* system call to fork a copy of the server for each client. Here is the code included from the previous article. This time it is for IPv4.

```
1.#include <stdio.h>
2.#include <unistd.h>
3.#include <sys/types.h>
4.#include <sys/socket.h>
5.#include <netinet/in.h>
6.#include <signal.h>

7.int main()
```

```
8.{
9.    int sfd, cfd;
10.   socklen_t len;
11.   char ch, buff[INET_ADDRSTRLEN];
12.   struct sockaddr_in saddr, caddr;

13.   sfd= socket(AF_INET, SOCK_STREAM, 0);

14.   saddr.sin_family=AF_INET;
15.   saddr.sin_addr.s_addr=htonl(INADDR_ANY);
16.   saddr.sin_port=htons(1285);

17.   bind(sfd, (struct sockaddr *)&saddr, sizeof(saddr));

18.   listen(sfd, 5);
19.   signal(SIGCHLD, SIG_IGN);
20.   while(1) {
21.       printf("Server waiting\n");
22.       len=sizeof(caddr);
```